NexusResidencial — Documento técnico y de negocio

Cómo funciona cada parte del sistema, por qué se eligió cada tecnología, cómo se protegen los datos y cuánto cuesta operarlo

Denil Parada · Diana Sosa · Diego Reyes · Arodi Chávez · Emilio Chen · Luis Estrada
Ingeniería de Software 1 · Sección 40 · Noviembre 2026

- 1. Qué es NexusResidencial (en una página)
- 2. Stack tecnológico y por qué se eligió cada pieza
- 3. Cómo funciona cada módulo (la lógica por dentro)
- 4. Seguridad y datos sensibles
- 5. Pruebas de experiencia de usuario (Neurolab UVG)
- 6. Escalabilidad: ¿cuántos usuarios aguanta?
- 7. Varios residenciales: ¿cómo se trabaja la base de datos?
- 8. Costos de operación y precios de servicios externos
- 9. Costo de desarrollo (método de Puntos de Casos de Uso del curso)
- 10. Modelo de venta, ganancia y punto de equilibrio
- 11. Valor e impacto (situación actual vs NexusResidencial)
- 12. Preguntas frecuentes del jurado (respuestas cortas)

1. Qué es NexusResidencial (en una página)

NexusResidencial es un software web, instalable en el celular, para administrar residenciales y condominios en Guatemala. Une en un solo sistema lo que hoy se hace con libretas, Excel, llamadas y WhatsApp: control de visitas con código QR, cobros y estados de cuenta, reservas de amenidades, comunicados, sanciones, reportes de fallas, votaciones y reportes para la administración.

Pregunta	Respuesta corta
¿A quién se le vende?	A la **administración** de residenciales y condominios (o a la empresa administradora). Es quien paga la suscripción.
¿Quién lo usa?	Cuatro perfiles: **administrador**, **residente** (propietario), **inquilino** y **guardia** de garita. El administrador además puede crear roles propios (contador, junta directiva, recepción…).

Pregunta	Respuesta corta
¿Con quién se validó?	Con **entrevistas de Design Thinking** a administradores, residentes, inquilinos y guardias (enero–febrero 2026) y con **pruebas de usabilidad en el Neurolab UVG** (15 de octubre de 2026).
¿Cómo empezó?	Como proyecto para un *product owner*. A mitad del curso el enfoque cambió a **producto de software para vender** a cualquier residencial; por eso el sistema permite una copia por cliente con su propia marca.
¿Por qué ahora?	Los condominios y la vivienda vertical son la tendencia dominante de construcción en Guatemala (Revista E&N, 2025); cada nuevo complejo necesita administrar accesos, cobros y convivencia.
¿En qué estado está?	En producción en `nexus-residencial.com` (frontend) y `api.nexus-residencial.com` (backend), desplegado en Railway. Pagos en línea en **modo pruebas (sandbox) de Recurrente**; listo para pasar a cobro real cuando el cliente tenga su cuenta comercial.

Línea del tiempo del proyecto

Etapa	Fecha	Resultado
Empatía (brief, usuarios, AEIOU)	20 ene 2026	4 perfiles de usuario definidos
Empatía y definición (entrevistas a los 4 perfiles)	2 feb 2026	Entrevistas, mapas de empatía, insights, “¿Cómo podríamos…?”
Ideación (Design Studio, sketches)	16 feb 2026	Ideas por perfil y restricciones
Prototipos, historias y casos de uso	16 mar 2026	Backlog inicial
Sprints 1 → 10 (Scrum, Jira)	16 mar → 27 oct 2026	11 sprints registrados en Jira, **promedio de 51.09 puntos por sprint**
Neurolab (UX con seguimiento ocular y GSR)	15 oct 2026	SUS 69.6, 94 % de tareas completadas, hallazgos H1–H6
Producto final para venta	nov 2026	Copia por cliente, marca propia, roles, indicadores, comunidad, importación, app instalable

2. Stack tecnológico y por qué se eligió cada pieza

Capa	Tecnología	Por qué la elegimos	Alternativas consideradas
Interfaz	**React 18 + TypeScript + Vite + Tailwind CSS**	Un solo lenguaje (JavaScript/TypeScript) en todo el equipo; componentes reutilizables entre los 4 roles; Vite compila en segundos; TypeScript evita errores de datos entre pantallas.	Angular (más pesado y curva mayor), HTML + jQuery (difícil de mantener con 70 pantallas).
Servidor	**Node.js + Express 5**	Mismo lenguaje que el frontend; muy liviano; el equipo ya lo dominaba; ecosistema para PDF, Excel, correo y pagos.	Django (Python) o Spring (Java): más curva de aprendizaje para el equipo.
Base de datos	**MySQL 8 (InnoDB)**	Los datos son relacionales (casas → contratos → cuotas → pagos) y necesitan **transacciones** e integridad referencial: un pago no puede quedar a medias.	MongoDB: no garantiza esas relaciones de forma natural.
Hospedaje	**Railway** (backend, frontend y MySQL)	Despliegue automático al hacer *merge* a `main`; MySQL administrado; se paga por uso real (~USD 10–25 al mes por residencial pequeño).	AWS (más complejo de operar), un VPS (mantenimiento manual), Vercel (no ofrece MySQL ni procesos de fondo).
Pagos	**Recurrente** (pasarela guatemalteca)	Cobra en quetzales con tarjetas locales, **sin mensualidad**, con página de pago alojada por Recurrente (los datos de tarjeta nunca pasan por nuestro servidor).	Stripe (no opera comercios en Guatemala), PayPal (USD y más fricción), POS bancario (no sirve en línea).
Mapa 3D	**Three.js**	Dibuja el residencial en 3D en el navegador, sin plugins.	Imágenes estáticas (no reflejan el estado real de cada casa).
Gráficas	Recharts	Gráficas de indicadores en React.	Chart.js.

Capa	Tecnología	Por qué la elegimos	Alternativas consideradas
Documentos	PDFKit y ExcelJS	Comprobantes y reportes generados en el servidor.	Generar en el navegador (se pierde control del formato).
Correo	Nodemailer + Gmail (SMTP)	Gratis para el volumen actual.	Resend/SendGrid cuando crezca el volumen (ver sección 8).
Avisos al celular	Web Push (estándar del navegador, llaves VAPID)	Gratis y sin depender de Firebase.	Firebase Cloud Messaging.
Calidad	Jest, node:test, Supertest, Vitest, Testing Library, Playwright	Pruebas automáticas en cada capa.	—

Tamaño del sistema: unas 50,000 líneas de código, 67 tablas en la base de datos, 233 archivos de pruebas; **920 + 132 pruebas de backend y 575 de frontend pasando**; 67 pantallas entre los 4 roles.

Arquitectura

```
 Celular / computadora (React, app instalable)
        │  HTTPS + token de sesión firmado
        ▼
API Node.js / Express  ── valida rol y permisos en CADA petición
        │  consultas parametrizadas
        ▼
MySQL (InnoDB, transacciones)
        │
        ├─► Recurrente (pagos)   ◄── webhook firmado de confirmación
        ├─► Gmail SMTP (correos de avisos)
        └─► Servicio push del navegador (avisos al celular)
```

3. Cómo funciona cada módulo (la lógica por dentro)

3.1 Mapa 3D de viviendas: cómo se dibuja y cómo escala

1. **Los datos vienen de la base, no del dibujo.** Cada vivienda tiene `torre` (manzana), `numero`, y opcionalmente `mapa_fila` y `mapa_columna`.
2. **Una función pura convierte viviendas en posiciones** (`buildHouseMapLayout`): agrupa por manzana; cada manzana es un bloque de lotes de 6.5 × 8 unidades (1 unidad ≈ 2 m); las filas pares miran a la calle norte y las impares a la sur; los bloques se acomodan de dos en dos separados por calles.

3. **Si una casa no tiene posición guardada**, se coloca sola en el primer hueco libre de su manzana. Por eso el mapa **escala solo**: agregar 10 o 200 casas no requiere tocar código; el mundo, la sombra, la niebla y la cámara se recalculan al tamaño del residencial.
4. **El motor 3D (Three.js)** construye la escena: cielo, día/noche, faroles, muro, parque, casa club, autos y un modelo de casa según el "modelo" de la vivienda. El color indica el estado: verde = disponible, naranja = ocupada, gris = inactiva.
5. **Interacción:** al tocar una casa, un *raycaster* detecta qué lote se tocó y la cámara "vuela" hacia ella con una animación. Vista 3D o aérea, día o noche.
6. **Si el equipo no soporta 3D (sin WebGL)**, se muestra automáticamente el **mapa 2D** con la misma información. El 3D se carga bajo demanda, para no hacer lenta la primera pantalla.

3.2 Visitas y códigos QR

Cómo se crea el QR

1. El residente (o inquilino) llena: nombre, DPI o placa, fecha, hora de inicio y hora de fin.
2. El servidor valida: que la fecha y horas sean correctas, que la visita **no cruce la medianoche**, que esté dentro del **horario de visitas** del residencial (por defecto 06:00–22:00, días habilitados y duración máxima configurables por el administrador).
3. Se genera un **token aleatorio de 36 caracteres** (`crypto.randomBytes(18)`, 144 bits: imposible de adivinar) y se guarda en la tabla `ACCESO` junto con la visita.
4. El código QR que ve el residente es solo el texto `NEXUSVISIT:<token>` dibujado en el navegador. **El QR no contiene datos personales**: si alguien lo fotografía, no obtiene nombre, casa ni DPI.

Cómo valida el guardia

1. Escanea con la cámara (lectura con la librería jsQR) o escribe el código.
2. El servidor busca el token y calcula el estado **en ese momento, con la hora de Guatemala**:

Estado	Cuándo	Qué pasa en garita
Aún no vigente	Antes de la hora de inicio	Rechazado: "Espere hasta la hora de inicio programada"
Vigente	Entre inicio y fin	**Se registra el ingreso** en el mismo paso
Ya utilizado	Ya tiene ingreso	Rechazado: "Este QR ya fue utilizado"
Expirado	Después de la hora de fin	Rechazado: "QR expirado"
Cancelado	El residente la canceló	Rechazado: "Este QR ya no es válido"
Pendiente de aprobación	Acceso de emergencia sin aprobar	Rechazado hasta que la administración apruebe

3. **Cada intento queda registrado**, aceptado o rechazado, con el guardia que lo hizo (auditoría).

4. **Dos guardias escaneando el mismo QR al mismo tiempo:** la base de datos hace un `UPDATE … WHERE estado = 'AUTORIZADA'`; solo uno lo logra y el otro recibe "ya utilizado". No hay doble ingreso.

Registro de salida: el guardia busca la visita (en "Visitas recientes" o escaneando de nuevo) y marca la salida. Solo se puede registrar salida si antes hubo ingreso, y solo una vez. El panel del guardia muestra cuántas visitas están "Dentro ahora".

3.3 ¿Qué pasa si…? (casos de visitas que preguntan los jurados)

Situación	Qué hace el sistema hoy
La visita dijo 9 pm, pero se queda hasta las 12	Su ingreso ya está registrado; el QR no se vuelve a usar. La salida **se puede registrar a cualquier hora** (no se bloquea), y en el historial queda la hora de fin autorizada y la hora real de salida, así que el administrador ve el exceso. Mientras no salga, aparece en "Dentro ahora" del guardia. *Mejora propuesta:* alerta automática al guardia y al residente cuando una visita pasa su hora de fin sin salida.
La visita llega después del rango que se puso	El QR está **expirado** y garita lo rechaza. El residente crea una nueva autorización en segundos desde su celular (si aún está dentro del horario de visitas).
Necesito un QR fuera del horario (6 am–10 pm)	No se puede crear una visita normal fuera de horario. Para urgencias existe **"Solicitar acceso de emergencia"**: el residente escribe el motivo, la administración recibe la alerta, lo aprueba o rechaza, y solo entonces el QR funciona. El guardia ve la solicitud como "Acceso especial pendiente".
Una visita que cruza la medianoche (23:00 → 02:00)	Se rechaza con un mensaje claro: debe empezar y terminar el mismo día; si es urgente, usar acceso de emergencia.
Le paso el QR a otra persona	El QR solo sirve una vez y solo en su horario; el guardia ve en pantalla el nombre, DPI y placa autorizados y compara con quien llega.
Me roban el celular	El usuario (o el administrador) cierra esa sesión desde "Sesiones activas" y el token deja de funcionar al instante.

3.4 Cobros, estados de cuenta, recargos y sanciones

- **Contrato por vivienda:** compra al contado, compra financiada (enganche, plazo y cuota calculada automáticamente) o renta. El día de pago se toma del día del contrato.
- **Facturación mensual automática:** el día 1 de cada mes se generan las cuotas de cada vivienda (mantenimiento, cuota de vivienda o alquiler), con una "fotografía" de las reglas vigentes, para

que un cambio de tarifa no altere meses anteriores.
- **Recargos y sanciones calculados por el sistema:** el administrador configura las reglas (día límite, porcentaje o monto mínimo) y con un clic aplica los recargos y genera las sanciones de todas las cuotas vencidas; el sistema evita duplicados y deja todo auditado. *Lo automático en segundo plano* es la facturación del día 1 y los recordatorios de pago y de reservas.
- **Recordatorios** por campana, correo y push antes del vencimiento.
- **Pago en línea:** el residente toca "Pagar", el servidor crea el cobro en Recurrente por el monto exacto pendiente y lo redirige a la página de pago de Recurrente. Al pagar, Recurrente avisa a nuestro servidor con un **webhook firmado**; el servidor verifica la firma, aplica el pago **una sola vez** (idempotente, aunque el aviso llegue repetido) y genera el comprobante.
- **Comprobante PDF** con la marca del residencial; **conciliación** administrativa contra Recurrente y **reembolsos**.

3.5 Roles y permisos (incluye roles personalizados)

- Cada petición pasa por un *middleware* que verifica el rol en el servidor (no basta con ocultar botones).
- **Roles personalizados:** el administrador crea, por ejemplo, "Contador" y elige por módulo: sin acceso, solo ver, o ver y editar. El servidor valida cada ruta contra esos módulos. Respaldos, restauraciones y la gestión de roles quedan siempre reservados al administrador principal, y un rol con acceso a Usuarios **no puede crear administradores** (no se puede escalar privilegios).

3.6 Otros módulos

Módulo	Cómo funciona
Amenidades y reservas	Horario por amenidad en bloques de 15, 30 o 60 minutos; validación de choques; límite de reservas activas por usuario; cancelación con historial.
Reportar fallas	Foto tomada desde el celular, comprimida en el navegador (máx. 1280 px, JPEG) antes de subirse; tablero para la administración (abierto → en proceso → resuelto) con aviso al residente en cada cambio.
Votaciones	**Un voto por vivienda** (llave única vivienda + votación en la base); vota el residente responsable; resultados ocultables hasta el cierre.
Documentos y directorio	Reglamentos, actas y estados financieros descargables; teléfonos de emergencia (PNC 110, Bomberos 122/123, Cruz Roja 125) con llamada de un toque.
Indicadores	Morosidad mes a mes, ocupación de amenidades, visitas por día, fallas reportadas vs resueltas.

Módulo	Cómo funciona
Auditoría	Quién hizo qué, cuándo, desde qué IP, con datos antes/después.
Importar desde Excel	Plantilla de 4 hojas (viviendas, residentes, inquilinos, saldos); vista previa fila por fila con errores; no se guarda nada hasta confirmar.
Primeros pasos	Asistente de 6 pasos para dejar listo un residencial nuevo en minutos.
App instalable y push	PWA: se instala desde el navegador (Android con botón; iPhone desde Compartir → Agregar a inicio). Los avisos llegan aunque la app esté cerrada.

4. Seguridad y datos sensibles

Qué datos sensibles maneja el sistema: contraseñas, DPI y placas de visitantes, datos de contacto, saldos y pagos, documentos del residencial y fotos de fallas. **No guarda datos de tarjetas.**

Riesgo	Cómo lo protegemos
Robo de contraseñas	Se guardan cifradas con **bcrypt** (nunca en texto). Cambiar la contraseña cierra las demás sesiones.
Adivinar contraseñas	**Límite de intentos:** 20 inicios de sesión, 5 recuperaciones y 5 cambios de contraseña cada 15 minutos.
Robo de sesión	Token firmado con **HMAC-SHA256** que vence en 8 horas, **más** una tabla de sesiones activas en el servidor: cada dispositivo se puede cerrar al instante. **Cierre automático por inactividad** a los 30 minutos con aviso previo.
Recuperación de contraseña	Enlace de un solo uso que **vence en 30 minutos**; en la base se guarda solo el hash SHA-256 del token.
Ver datos de otra casa	Cada consulta filtra por la vivienda del usuario en el servidor; un residente nunca puede pedir datos de otro cambiando un número en la URL.
Inyección SQL	Todas las consultas son **parametrizadas** (`?`), nunca se concatena texto del usuario.
Ataques al navegador	Cabeceras de seguridad con **Helmet**, CORS restringido al dominio del frontend, límite de tamaño de peticiones.
Pagos	La tarjeta se escribe en la página de **Recurrente**, no en la nuestra; el webhook se verifica con firma (Svix); los

Riesgo	Cómo lo protegemos
	pagos son idempotentes.
Llaves y contraseñas del sistema	Solo en **variables de entorno de Railway**, nunca en el repositorio. Conexión siempre por **HTTPS**.
Errores y registros	Los mensajes al usuario no revelan detalles internos; el registro de errores elimina datos sensibles.
Pérdida de información	**Respaldos automáticos** y restauración validada; auditoría de cambios.
Privacidad	Política de privacidad publicada en `/privacidad`. El QR no contiene datos personales. Cada residencial tiene **su propia base de datos** (sección 7).

---

5. Pruebas de experiencia de usuario (Neurolab UVG)

Protocolo: 15 de octubre de 2026, 10:15–11:30, Neurolab UVG (CIT 415-B). 3 roles (administrador, guardia, residente), 6 voluntarios (2 por rol), **seguimiento ocular, sensor GSR Shimmer** (estrés), cuestionario **SUS** y entrevista breve.

ID	Rol	Tiempo	Tareas	Errores	Pidió ayuda	SUS
PUX-01	Administrador	5:20	3/3	1	No	80.0
PUX-02	Guardia	5:10	3/3	2	No	72.5
PUX-03	Residente	9:40	2/3	4	Sí	57.5
PUX-04	Administrador	5:45	3/3	2	No	75.0
PUX-05	Guardia	5:20	3/3	3	No	67.5
PUX-06	Residente	6:15	3/3	3	No	65.0
Promedio		**6:15**	**17/18 (94 %)**	**2.5**	1 de 6	**69.6**

Hallazgos y qué hicimos

#	Hallazgo	Estado en la versión final
H1	Navegación dependiente del menú; nombres parecidos (Visitas, Accesos, Notificaciones)	**Parcial:** menú agrupado por secciones (General, Mi unidad, Finanzas, Comunidad) y accesos directos desde el panel.
H2	La pre-autorización de visitas era difícil de encontrar (única tarea no completada sin ayuda)	**Atendido:** botón principal **"Autorizar una visita"** en el panel del residente y del inquilino.

#	Hallazgo	Estado en la versión final
H3	El filtro del historial del guardia era pequeño y lejos de la tabla	**Atendido:** filtros grandes (fecha, estado, búsqueda) justo encima de la tabla.
H4	Jerarquía visual poco clara en el panel del residente	**Parcial:** tarjetas de saldo, visitas, reservas y avisos con acción directa y botón principal.
H5	Confirmaciones poco visibles	**Atendido** en el Sprint 10 (ventanas de confirmación claras en toda la app).
H6	El rol administrador funciona bien	Se mantuvo como referencia de diseño.
UX-05	Usuarios pidieron modo oscuro	Pendiente en el backlog.

Muestra a usuarios finales (Sprint 10): dijeron que la plataforma les gustaba, que las opciones se encontraban fácilmente y que se entendía cada pantalla. **Lección:** la encuesta dijo "es fácil"; el Neurolab mostró *dónde* se pierde el residente. Las dos fuentes se complementan. *Siguiente paso:* repetir la prueba para medir si H2 y H3 subieron el SUS del residente.

6. Escalabilidad: ¿cuántos usuarios aguanta?

Prueba de carga real (herramienta autocannon, servidor en una sola instancia de Node.js, 4 vCPU, base MySQL local):

Escenario	Conexiones simultáneas	Peticiones por segundo	Tiempo de respuesta (97.5 %)	Errores
Página pública (marca)	50	~2,900	28 ms	0
Con sesión (notificaciones)	50	~280	223 ms	0
Consulta más pesada (estado de cuenta)	100	~210	579 ms	0

Cómo se traduce a personas: un usuario activo hace en promedio una petición cada ~15 segundos (navegar + actualizaciones automáticas). Con ~280 peticiones por segundo, **una sola instancia atiende del orden de 3,000–4,000 usuarios activos al mismo tiempo.** Un residencial de 300 casas (~900 usuarios) con 30 % conectados a la vez son ~270 usuarios: menos del 10 % de la capacidad.

Si crece: Railway permite subir recursos de la instancia o agregar réplicas sin cambiar código; la base se puede agrandar con un clic. *Nota honesta:* la prueba se hizo en el entorno de desarrollo, no en Railway; los números reales dependen del plan contratado.

---

7. Varios residenciales: ¿cómo se trabaja la base de datos?

Modelo elegido: una copia por cliente (opción A).

- Cada residencial tiene **su propio proyecto en Railway**: su frontend, su backend y **su propia base de datos MySQL**, con su dominio (`brisas.nexus-residencial.com`) y su marca (nombre, logo, color en pantallas, correos y PDF).
- **Todos usan el mismo código** (la rama `main`). Una mejora se publica una vez y llega a todos.
- **Si un cliente pide un cambio que el otro no quiere:**
    1. Si es configurable (horarios, tarifas, reglas de recargo, amenidades, roles, marca), **cada uno lo configura** desde su panel: no se toca código.
    2. Si es una función nueva de interés general, se construye **como opción activable** y cada cliente decide si la usa.
    3. Si es algo muy particular, se cotiza como desarrollo a la medida y se entrega igualmente como opción configurable.
    4. **Nunca se hace una copia distinta del código por cliente**: eso multiplicaría el mantenimiento y los errores.
- **Ventajas:** los datos de un residencial nunca se mezclan con otro (seguridad), cada uno se respalda y restaura por separado, y un problema en uno no afecta a los demás.
- **Desventaja:** hay que operar N despliegues. Con más de ~30 clientes conviene migrar a un modelo *multi-tenant* (una base con identificador de residencial) para bajar costos.
- **Alta de un cliente nuevo:** se crea el proyecto, se ponen sus variables, el sistema arranca vacío con su administrador, y el asistente de **Primeros pasos** + la **importación desde Excel** lo dejan operando el mismo día (guía en `docs/despliegue/NUEVO-RESIDENCIAL.md`).

---

8. Costos de operación y precios de servicios externos

Servicio	Costo	Notas
Railway (por residencial)	**~USD 10–25 al mes** (≈ Q80–Q195)	Pago por uso: USD 10 por GB de RAM al mes, USD 20 por vCPU al mes, USD 0.15 por GB de disco. Plan Pro USD 20/mes por cuenta, incluye USD 20 de uso.
Recurrente (pagos reales)	**4.5 % + Q2 por transacción exitosa** (cuenta comercial)	Sin mensualidad. Cuenta personal: 5.25 % + Q2. **El sandbox (modo pruebas) no cobra nada**: no mueve

Servicio	Costo	Notas
		dinero real. Ejemplo: cuota de Q500 → comisión de Q24.50.
Gmail (correos)	**Gratis**	No cobra por correo. Límite de ~500 destinatarios al día (cuenta normal) o 2,000 (Google Workspace). Un comunicado a todos cuenta un correo por persona.
Alternativa de correo	Resend: gratis hasta 3,000/mes (100/día); USD 20/mes por 50,000	Para residenciales grandes o muchos clientes.
Avisos push	**Gratis**	Estándar del navegador.
Dominio	~Q100–Q150 al año	Un dominio para todos los clientes con subdominios.
Almacenamiento de PDF	**Q0 adicional**	Comprobantes y reportes **no se guardan**: se generan al momento desde la base. Documentos de comunidad (máx. 3 MB) y fotos comprimidas se guardan en MySQL; si el volumen crece, se pasan a almacenamiento de objetos (S3/R2).

Si se acaba el límite de Gmail: los correos quedan en cola y se reintentan (hasta 24 horas); la campana dentro de la app y los avisos push siguen funcionando, porque no dependen del correo.

---

9. Costo de desarrollo (método de Puntos de Casos de Uso del curso)

1) Actores (FPA): 4 personas con interfaz gráfica (complejos, 3 c/u) = 12; Recurrente (API, simple) = 1; servidor de correo (protocolo, medio) = 2; servicio push (API, simple) = 1 → **FPA = 16**.

2) Casos de uso (FPCU)

Complejidad	Casos de uso	Peso	Total
Complejo (más de 7 transacciones)	Autorizar visita con QR; validar QR e ingreso/salida; pagar en línea; conciliación y reembolsos; contrato y facturación mensual; reservar amenidad;	10 × 15	150

Complejidad	Casos de uso	Peso	Total
	recargos y sanciones automáticos; roles personalizados; importar desde Excel; respaldos y restauración		
Medio (4 a 7)	Usuarios; viviendas y mapa 3D; accesos de emergencia; reportes PDF/Excel; indicadores; recordatorios; fallas; votaciones; comunicados; auditoría; sesiones activas; proveedores; usuarios autorizados	13 × 10	130
Simple (menos de 4)	Iniciar/cerrar sesión; recuperar contraseña; notificaciones; configuración y marca; documentos y directorio; estado de cuenta; primeros pasos	7 × 5	35
		FPCU	**315**

PCU = FPA + FPCU = 16 + 315 = 331

3) Factor de complejidad técnica (FCT) — valores asignados: T1 distribuido 3, T2 rendimiento 3, T3 eficiencia usuario 4, T4 procesamiento complejo 4, T5 reutilizable 3, T6 instalación 4, T7 facilidad de uso 5, T8 portabilidad 3, T9 facilidad de cambio 4, T10 concurrencia 4, T11 seguridad 5, T12 acceso a terceros 3, T13 entrenamiento 2. Σ(peso × valor) = 48.5 → **FCT = 0.6 + 0.01 × 48.5 = 1.085**

4) Factor de ambiente (FA) — E1 familiaridad con Scrum 3, E2 experiencia en el dominio 3, E3 orientación a objetos 3, E4 analista líder 3, E5 motivación 5, E6 estabilidad de requisitos 2 (el enfoque cambió de *product owner* a producto para vender), E7 personal a tiempo parcial 3, E8 dificultad del lenguaje 2. Σ = 14.5 → **FA = 1.4 − 0.03 × 14.5 = 0.965**

5) PCUA = 331 × 1.085 × 0.965 = 346.6

6) Factor de conversión: factores bajo la media (E1–E6) = 1 (E6) y sobre la media (E7–E8) = 0 → suma 1 ≤ 2 → **FC = 20 horas-hombre**. **Esfuerzo E = 346.6 × 20 ≈ 6,931 horas-hombre ≈ 43.3 hombres-mes.**

Actividad	%	Horas-hombre
Análisis	10	693
Diseño	20	1,386

Actividad	%	Horas-hombre
Implementación	40	2,772
Pruebas	15	1,040
Otras actividades	15	1,040
Total	100	**6,931**

Tiempo: 6,931 h ÷ 6 personas = 1,155 h por persona ≈ 7.2 meses de tiempo completo, coherente con los sprints de marzo a noviembre más las fases de Design Thinking. **Costo:** THP = Q20,000 ÷ 160 h = Q125/h; K = 1.5 (costos indirectos) → **C = 6,931 × 1.5 × Q125 ≈ Q1,299,600**. Coincide con el costo que el equipo reportó en los informes de sprint: **6 desarrolladores × Q20,000 × 11 meses = Q1,320,000**.

---

10. Modelo de venta, ganancia y punto de equilibrio

Mercado de referencia: Hestia (gratis hasta 15 unidades, cobra en quetzales), TuCondominio (USD 20/mes hasta 50 unidades), Condóminos (USD 15 por predio/mes), LAYERS (desde ~USD 1.15 por unidad/mes). La mayoría cobra **por unidad o por condominio**.

Precio propuesto para NexusResidencial

Concepto	Precio
Instalación y configuración (una vez)	Q2,500 (alta, marca, importación de Excel y capacitación)
Suscripción	**Q10 por vivienda al mes** (≈ USD 1.30), mínimo Q800/mes

Ejemplo: residencial de 150 casas

Concepto	Mensual	Anual
Ingreso por suscripción	Q1,500	Q18,000
Railway	−Q155	−Q1,860
Correo (Gmail gratis; Workspace si se necesita)	Q0 a −Q55	Q0 a −Q660
Soporte (4 h × Q125)	−Q500	−Q6,000
Ganancia	**≈ Q790–Q845 (53–56 %)**	**≈ Q9,500–Q10,100**

Las comisiones de Recurrente (4.5 % + Q2) las paga el residencial sobre lo que cobra en línea, no NexusResidencial.

Escenarios anuales (150 casas promedio por cliente, ganancia ≈ Q820/mes por cliente)

Clientes	Ingreso anual	Costo de operación anual	Ganancia anual
1	Q18,000	Q8,160	Q9,840
10	Q180,000	Q81,600	Q98,400
30	Q540,000	Q244,800	Q295,200
60	Q1,080,000	Q489,600	Q590,400

Punto de equilibrio contra el costo de desarrollo (Q1,320,000): con ~30 clientes se recupera en unos 4.5 años; con 60 clientes, en poco más de 2 años. **Si se cuenta como pérdida:** con un solo cliente la operación sí deja ganancia mensual, pero no recupera el desarrollo; el negocio depende de vender a varios residenciales, y el modelo de copia por cliente permite sumar uno nuevo el mismo día.

11. Valor e impacto (situación actual vs NexusResidencial)

Proceso	Antes (residenciales hoy: libreta, Excel, llamadas, WhatsApp)	Con NexusResidencial	Métrica
Autorizar una visita	Llamada o WhatsApp al guardia, que anota en libreta; el guardia no puede verificar que el residente autorizó	El residente autoriza desde el celular; el guardia valida el QR en segundos	Tareas del Neurolab completadas en 6:15 min en promedio para 3 tareas completas
Cobros y morosidad	El administrador calcula vencimientos y multas a mano	Cuotas y recordatorios automáticos; recargos y sanciones calculados con un clic	Morosidad visible mes a mes en Indicadores
Trazabilidad	“¿Quién entró el martes a las 10 pm?” no tiene respuesta	Cada ingreso, pago y cambio queda registrado	100 % de los accesos con registro
Amenidades	Reservas por mensaje, choques y sin responsable	Calendario con validación de choques	0 reservas duplicadas

Supuestos: residencial de ~150 casas, ~40 visitas diarias, un administrador. Si cada visita toma 3 minutos de coordinación telefónica y ahora menos de 1 minuto, se ahorran ~1.5 horas diarias de garita; el cálculo de recargos en Excel (~8 h al mes) pasa a minutos.

Beneficiarios: administración (menos trabajo manual, información confiable), residentes e inquilinos (autonomía y transparencia), guardias (validación rápida y segura). **Alternativas:** seguir con

Excel/WhatsApp (gratis pero sin control), software extranjero genérico (sin pagos locales en quetzales ni QR de garita), o desarrollo propio (costoso).

---

12. Preguntas frecuentes del jurado (respuestas cortas)

- **¿Es para un residencial o para vender?** Para vender a cualquier residencial o empresa administradora. El proyecto empezó para un *product owner*; a mitad del curso cambió a producto de software, y por eso cada cliente tiene su propia copia con su marca.
- **¿Qué pasa si cae el internet en garita?** Sin conexión el QR no se puede validar contra el servidor; garita aplica el protocolo manual del residencial y luego la administración regulariza. *Mejora futura:* modo sin conexión con validación diferida.
- **¿Se puede hackear el QR?** El token tiene 144 bits aleatorios, sirve una vez y solo en su horario; no lleva datos personales.
- **¿Guardan tarjetas?** No. El pago ocurre en la página de Recurrente.
- **¿Cuánto cuesta operarlo?** ~USD 10–25 al mes de servidor por residencial; correo y push gratis al volumen actual.
- **¿Cuántos usuarios soporta?** Miles de usuarios activos por instancia según la prueba de carga; escala agregando recursos.
- **¿Qué aprendieron del Neurolab?** Que la encuesta decía “fácil”, pero el residente no encontraba la pre-autorización; lo cambiamos.

Fuentes (consultadas en noviembre 2026): revistaeyn.com (auge de la vivienda vertical en Guatemala, 2025); recurrente.com/precios; docs.railway.com/pricing; knowledge.workspace.google.com (límites de Gmail); resend.com/pricing; comparasoftware.gt, hestia.com.gt, tucondominio.online, condominos.app, layers.lat.